

Reviewer Pack — Dual Code Distance Lower Bound for Tseitin Refutations

Entry 8b11c41c9207 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-29 03:54:00 UTC

Dual Code Distance Lower Bound for Tseitin Refutations

Entry ID: 8b11c41c9207 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-29 03:54:00 UTC

1. Conjecture

Field A (mathematical branch): Proof Complexity **Field B** (complexity object): Coding Theory

Statement:

The minimal size of a resolution refutation for Tseitin formulas on expander graphs is exponentially related to the dual distance of the linear code defined by the graph's incidence matrix.

Rationale (proposer's reasoning):

Tseitin formulas correspond to linear systems over $\text{GF}(2)$, whose solution spaces form linear codes. The dual distance of these codes governs their error-correcting capabilities. By linking refutation complexity to dual code parameters, we isolate the algebraic structure of the problem from combinatorial expansion properties, avoiding natural proofs and relativization barriers. This conjecture leverages coding-theoretic techniques (e.g., dual distance bounds) alongside proof complexity methods.

Taxonomy category: PROOF_COMPLEXITY_TSEITIN (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a56c2ca7d88dcb00

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    def gaussian_elimination(A, b):
        n = len(b)
        for i in range(n):
            max_row = i + max(range(i, n), key=lambda k: abs(A[k][i]))
            A[i], A[max_row] = A[max_row], A[i]
            b[i], b[max_row] = b[max_row], b[i]
            factor = A[i][i]
            for j in range(n):
                A[i][j] /= factor
            b[i] /= factor
            for k in range(n):
                if k != i:
                    factor = A[k][i]
                    for j in range(n):
                        A[k][j] -= factor * A[i][j]
                    b[k] -= factor * b[i]
        return A, b

    def matrix_multiply(A, B):
        n = len(A)
        C = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def matrix_inverse(A):
        n = len(A)
        I = [[1 if i == j else 0 for j in range(n)] for i in range(n)]
        for i in range(n):
            max_row = i + max(range(i, n), key=lambda k: abs(A[k][i]))
            A[i], A[max_row] = A[max_row], A[i]
            I[i], I[max_row] = I[max_row], I[i]
            factor = A[i][i]
            for j in range(n):
```

```

        A[i][j] /= factor
        I[i][j] /= factor
    for k in range(n):
        if k != i:
            factor = A[k][i]
            for j in range(n):
                A[k][j] -= factor * A[i][j]
                I[k][j] -= factor * I[i][j]

    return I

def dual_distance(A):
    n = len(A)
    A_inv = matrix_inverse(A)
    D = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            D[i][j] = sum(A_inv[k][i] * A_inv[j][k] for k in range(n))
    return min([sum(D[i]) for i in range(n)])

def generate_tseitin_instance(n):
    random.seed(seed)
    variables = [f'x{i}' for i in range(1, n+1)]
    clauses = []
    for i in range(1, n+1):
        clauses.append(f'{variables[i-1]}')
    for i in range(1, n+1):
        for j in range(i+1, n+1):
            clauses.append(f'~{variables[i-1]} | ~{variables[j-1]}')
    return variables, clauses

def resolution_refutation_size(variables, clauses):
    random.seed(seed)
    n = len(variables)
    A = [[0] * (n + 1) for _ in range(n + 1)]
    b = [0] * (n + 1)
    for clause in clauses:
        if ' | ' not in clause and ' ~' not in clause:
            continue
        literals = clause.split(' | ')
        literal_indices = [variables.index(l[1:]) + 1 if l.startswith('~') else variables.index(l[1:]) for l in literals]
        A[literal_indices[0]][literal_indices[1]] += 1
        A[literal_indices[1]][literal_indices[0]] += 1
        b[literal_indices[0]] += 1
        b[literal_indices[1]] += 1
    A, b = gaussian_elimination(A, b)
    return sum(abs(x) for x in b)

n_values = [5, 10, 15, 20, 30, 40]
refutation_sizes = []
dual_distances = []

for n in n_values:
    variables, clauses = generate_tseitin_instance(n)
    A = [[0] * (n + 1) for _ in range(n + 1)]
    for clause in clauses:
        if ' | ' not in clause and ' ~' not in clause:

```

```

        continue
    literals = clause.split(' | ')
    literal_indices = [variables.index(l[1:]) + 1 if l.startswith('~') else variables.index(l[1:]) for l in literals]
    A[literal_indices[0]][literal_indices[1]] += 1
    A[literal_indices[1]][literal_indices[0]] += 1
    dual_distance_value = dual_distance(A)
    refutation_size = resolution_refutation_size(variables, clauses)
    refutation_sizes.append(refutation_size)
    dual_distances.append(dual_distance_value)

metric_name = "Refutation Size / Dual Distance"
metric_value = sum(refutation_sizes) / len(refutation_sizes) / sum(dual_distances) / len(dual_distances)
instances_tested = len(n_values)
conjecture_holds = all(r >= d for r, d in zip(refutation_sizes, dual_distances))
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": metric_name,
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    mean_metric_value = sum(r['metric_value'] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r['metric_value'] - mean_metric_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not r['conjecture_holds'] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result['conjecture_holds'])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_44d298fb.py", line 141, in <module>

```

result = run_trial(seed)
^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_44d298fb.py", line 116, in run_trial
    dual_distance_value = dual_distance(A)
    ^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_44d298fb.py", line 67, in dual_distance
    A_inv = matrix_inverse(A)
    ^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_44d298fb.py", line 55, in matrix_inverse
    A[i][j] /= factor
ZeroDivisionError: division by zero

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with division by zero error, preventing data collection. The crash suggests potential issues with matrix inversion but does not directly confirm or refute the conjecture. | next: Investigate why the incidence matrix is singular for tested expanders; verify graph construction parameters

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	84085
2	propose	ollama_remote	qwen3:8b	0	0	90331
3	preregistration	ollama_remote	qwen3:8b	0	0	31537
4	novelty	ollama_remote	qwen3:8b	0	0	22709
5	novelty	ollama_remote	qwen3:8b	0	0	20262
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17077
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24332
8	judge	ollama_remote	qwen3:8b	0	0	25384

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 315718 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/8b11c41c9207.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/8b11c41c9207.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/8b11c41c9207.tar.gz` (if generated)